ЛАБОРАТОРНАЯ 3 · ВАРИАНТ «МУЗЫКАЛЬНАЯ КЛАВИАТУРА»

Таймеры и звук простыми словами

Как нажатие клавиши превращается в ноту, что делает таймер и как объяснить свой код на защите.

Что сделано в этой работе

Компьютерная клавиатура управляет звуком на учебном стенде. Цифры выбирают ноты, отдельные клавиши меняют октаву и длительность, а Enter запускает семь нот подряд.

Нота получается из быстро повторяющегося электрического сигнала. Его создаёт аппаратный таймер **TIM2**. А сколько времени должна звучать нота, программа отсчитывает отдельно по **SysTick**.

Главная мысль

TIM2 отвечает за высоту звука. Программа и SysTick - за длительность ноты и порядок нот. Компьютер выбирает действие, микроконтроллер выполняет его на стенде.

Что можно сказать в начале защиты

«Я сделал музыкальную клавиатуру с управлением через UART. Нажатия обрабатываются сразу, без Enter. Частоты нот заранее записаны в таблице. Таймер TIM2 формирует на PA5 прямоугольный сигнал нужной частоты, который поступает к звукоизлучателю через усилитель. Длительность ноты и переходы гаммы контролирует основной цикл по счётчику миллисекунд. Пока нота играет, программа продолжает принимать клавиши».

Как учить

Сначала прочитай объяснения и разбери пример с Ля 440 Гц. Затем отвечай на вопросы, закрыв ответы. В конце реши задачи и повтори последовательность показа на стенде. В памятке 80 вопросов: от основных до подробностей кода.

Памятка описывает текущую реализацию lab3. В README принят вариант 1 «Музыкальная клавиатура». Официальный список вопросов преподавателя не предоставлен.

1. Как запустить и какие клавиши нажимать

Подключи питание и **DBG USB**. Замкни переключатель **SA2.7**: он соединяет звуковой сигнал с усилителем. SA2.8 для воспроизведения этой программой не требуется.

Настройки терминала

UART-канал FT2232, Interface 1; **57600, 8N1, без flow control**. Выбери UTF-8, выключи Local echo и **Local line editing**. Последнее нужно, чтобы терминал отправлял каждую клавишу сразу.

Клавиша	Действие
1, 2, 3, 4, 5, 6, 7	До, ре, ми, фа, соль, ля, си
+ / -	Октава выше / ниже; диапазон 1..9
A / a	Длительность больше / меньше на 0,1 с
Enter	Семь нот подряд: до, ре, ми, фа, соль, ля, си
Другой символ	Сообщение об ошибке с кодом символа

Начальное состояние

После сброса звука нет. Выбрана **первая октава, номер 5 из 9**, и длительность **1,0 секунды**. Нажми 6: должна начаться Ля первой октавы, 440 Гц.

```
Note la, octave first (5/9), 440.000 Hz, duration 1.0 s
```

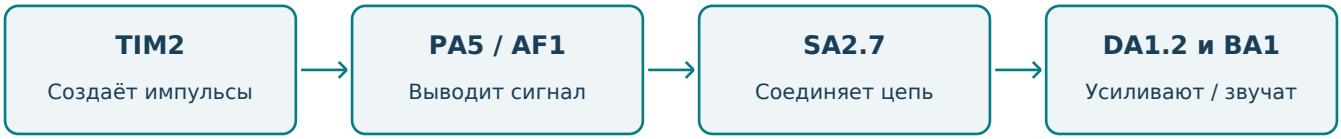
Ответ сообщает ноту, октаву, частоту и длительность. Эхо возвращает нажатый символ. Enter для отдельной ноты не нужен: здесь он запускает гамму.

Два важных ограничения

Длительность меняется от **0,1 до 5,0 с**. За пределом октав или времени настройка не меняется и выводится LIMIT. Для A/a нужна английская раскладка; большая и маленькая буквы выполняют разные действия.

2. Откуда на плате берётся звук

Звукоизлучатель превращает электрические изменения в колебания воздуха. В этой работе до него доходит повторяющийся сигнал от вывода PA5.



Сначала электрический сигнал, затем звук.

PA5 - вывод 5 порта A. **AF1** - настройка, которая подключает этот вывод к первому каналу TIM2. **DA1.2** - усилитель на схеме, **BA1** - звукоизлучатель.

Почему важен SA2.7

Если цепь разомкнута, сигнал может правильно формироваться на PA5, но не дойдёт до усилителя. Поэтому ответы в терминале ещё не доказывают, что вся звуковая цепь подключена.

Три разные характеристики

Что слышим или задаём	Что это означает
Высота	Как часто повторяется сигнал: выше частота - выше нота
Длительность	Сколько времени программа оставляет ноту включённой
Громкость	Насколько сильный звук; отдельной настройки громкости здесь нет

Например, 440 Гц - это 440 повторений сигнала за секунду. Длительность 1 с означает, что такая нота играет одну секунду. Можно оставить те же 440 Гц, но увеличить время до двух секунд.

Почему не переключать вывод вручную в цикле

Главный цикл также обрабатывает UART и сообщения. Ручные переключения зависели бы от времени выполнения этих дел. Аппаратный таймер создаёт импульсы самостоятельно, пока процессор занимается остальной программой.

Надпись сети T1_CH1/A5 на общей схеме - имя соединения. Для STM32F427 сочетание PA5/AF1 выбирает именно TIM2_CH1.

3. Частота, нота и октава

Что такое частота

Частота показывает количество повторений за секунду и измеряется в **герцах, Гц (Hz)**. Ля первой октавы здесь равна 440 Гц. Ля на октаву выше - 880 Гц, на октаву ниже - 220 Гц.

Переход на октаву выше удваивает частоту. Между соседними нотами до, ре, ми частота растёт по музыкальному правилу, а не прибавлением одного и того же числа герц.

Ноты первой октавы в таблице программы

Клавиша	Нота	Частота, Гц
1	До / do	261,626
2	Ре / re	293,665
3	Ми / mi	329,628
4	Фа / fa	349,228
5	Соль / sol	391,995
6	Ля / la	440,000
7	Си / si	493,883

Почему первая октава обозначена 5/9

Программа нумерует все доступные октавы с самой низкой. По порядку: субконтроктава, контроктава, большая, малая, первая, вторая, третья, четвёртая, пятая. Поэтому первая музыкальная октава стоит пятой в этом списке.

В терминале названия: `subcontra`, `contra`, `great`, `small`, `first`, `second`, `third`, `fourth`, `fifth`. Всего **9 × 7 = 63 частоты**.

Что такое миллигерцы в коде

Чтобы хранить дробную частоту целым числом, используют **миллигерцы (mHz)**. В одном герце 1000 миллигерц. Поэтому число `440000` в таблице означает 440 Гц, а не 440 тысяч герц.

Не путай обозначения: mHz - миллигерцы, MHz - мегагерцы, то есть миллионы герц.

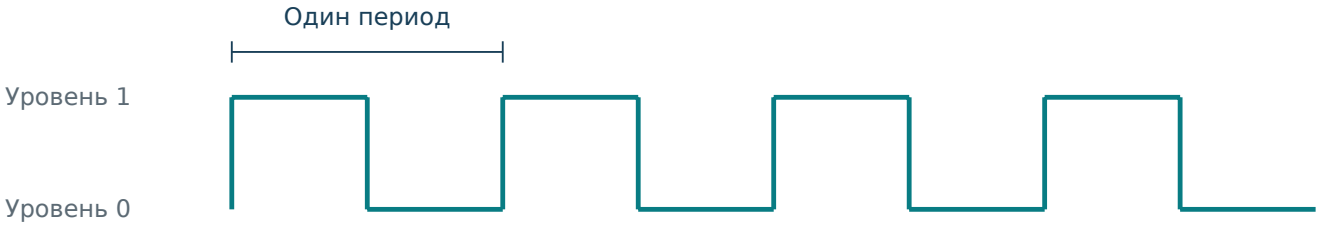
4. Как таймер делает повторяющийся сигнал

Таймер - счётчик, который работает сам

Представь быстро идущий счётчик. Он увеличивает число на каждом своём такте. Дошёл до заданного конца - вернулся к нулю и начал снова. Такой проход может задавать один период выходного сигнала.

Высокий и низкий участки

Выход некоторое время находится в 1, затем в 0. После этого последовательность повторяется. Такой способ формирования импульсов называется **ШИМ**, или **PWM**.



Одинаковые отрезки времени по горизонтали. Заполнение около 50 %.

Здесь высокий участок занимает примерно половину периода. Это называют **заполнением около 50%**. Частоту меняют для выбора ноты, а заполнение оставляют примерно одинаковым.

Как работает выбранный режим PWM1

Пока число в счётчике CNT меньше CCR1, выход высокий. Когда счётчик достигает CCR1, выход становится низким. После конца периода всё повторяется.

Что делает процессор во время этого

Процессор один раз задаёт настройки. Дальше TIM2 сам формирует фронты - переходы между 0 и 1. **Прерывания TIM2 для каждого импульса не нужны и в коде не включены.**

SysTick при этом продолжает считать миллисекунды. Когда длительность ноты истечёт, основной код остановит звук. Быстрые электрические импульсы и длительность музыкальной ноты - две разные задачи.

5. CNT, PSC, ARR и CCR1 без заучивания

У таймера есть специальные ячейки управления - **регистры**. Каждый отвечает за понятную часть работы.

Регистр	Что означает
CNT	Текущее число счётчика: где мы находимся внутри периода
PSC	Предделитель: замедляет поступающие такты в PSC + 1 раз
ARR	Последнее число счётчика перед возвратом к нулю
CCR1	Граница высокого участка для первого канала

Пример на маленьких числах

Пусть ARR=9 и CCR1=5. Счётчик проходит 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. Получается **10 тактов на период**, потому что ноль тоже считается.

На значениях 0..4 выход высокий - пять тактов. На 5..9 низкий - ещё пять. Значит, высокий участок занимает половину периода.

```
ARR = 9  ->  10 тактов на период
CCR1 = 5 ->  5 тактов высокого уровня
```

Почему PSC=0 не останавливает таймер

Делитель равен PSC+1. Если PSC=0, делим на 1: скорость не уменьшается. Если PSC=1, делим на 2. Для нот в этой работе выбран PSC=0.

Почему выбран 32-разрядный TIM2

Низкая частота требует длинного периода и большого числа тактов. Для самой низкой ноты нужно около 3,67 миллиона тактов. Число больше 65535 не поместится в 16-битный период без подбора другого предделителя.

У TIM2 счётчик и ARR 32-битные, поэтому все девять октав помещаются с одним PSC=0. Это упрощает настройку.

6. Расчёт на примере Ля 440 Гц

Сначала общий смысл

Таймер получает **60 000 000 тактов в секунду**. Нужно получить 440 периодов в секунду. Значит, один период должен занимать примерно 60 000 000 / 440 тактов.

```
K = round(60 000 000 / 440) = 136364
ARR = K - 1 = 136363
CCR1 = K / 2 = 68182
PSC = 0
```

Здесь K - число тактов в периоде. В коде результат называется `ticks`. Round означает округление до ближайшего целого. CCR1 задаёт половину периода для высокого уровня.

Общая формула

```
f = f_TIM / ((PSC + 1) * (ARR + 1))
```

f - частота выходного сигнала; f_TIM - частота, приходящая к таймеру. PSC+1 замедляет счёт, ARR+1 задаёт число отсчитанных тактов на период.

Почему частота получится чуть не ровно 440

Таймер считает целые такты. После округления период равен 136364 тактам, поэтому расчётная частота - **около 439,9988 Гц**. Это небольшая ошибка округления. Погрешность самого генератора HSI добавляется отдельно.

Как считаются миллигерцы

```
K = round(60 000 000 * 1000 / F_mHz)
```

F_mHz - частота из таблицы, например 440000. Множитель 1000 нужен из-за перехода к миллигерцам. Промежуточный результат 60 миллиардов не помещается в обычное 32-битное число, поэтому используется `uint64_t`.

Что запомнить для защиты

У 440 Гц: PSC=0, ARR=136363, CCR1=68182. Гораздо важнее сам смысл: длительность периода задаёт ARR, а CCR1 делит период на высокий и низкий участки.

7. Почему ядро 120 МГц, а TIM2 - 60 МГц

Источник и преобразование частоты

Внутренний генератор **HSI** даёт 16 МГц. Блок **PLL** с помощью делителей и умножителя создаёт частоту 120 МГц. Мегагерц - миллион тактов в секунду.

```
16 / 8 = 2 МГц
2 * 120 = 240 МГц
240 / 2 = 120 МГц для SYSCLK
```

Этим действиям соответствуют настройки PLL: M=8, N=120, P=2. Ядро получает 120 МГц. Ветви для периферии настроены отдельно.

Кому нужны такты	Как получаются	Частота
Ядро и AHB	120 МГц / 1	120 МГц
Шина APB1	120 МГц / 4	30 МГц
Таймер TIM2	2 × APB1 при этой настройке	60 МГц
Шина APB2 и USART1	120 МГц / 2	60 МГц

Откуда взялось удвоение у TIM2

В этой конфигурации TIMPRE=0. Если делитель APB больше единицы, таймер получает удвоенную частоту своей шины. APB1 равна 30 МГц, значит TIM2 получает 60 МГц. Для USART1 такого удвоения нет.

Что пришлось перенастроить после ЛР 2

SysTick по-прежнему должен давать 1 мс: при 120 МГц LOAD=119999. USART1 на шине 60 МГц при 57600 бод получает BRR=1042. Старый делитель UART от 16 МГц уже не подходит.

Шины имеют допустимые пределы частот, поэтому им заданы делители. Для Flash в этой реализации выбраны 3 такта ожидания, а для питания - scale 1. Это часть корректного запуска на 120 МГц.

8. Как одна нота заканчивается и начинается следующая

Длительность измеряется отдельно от частоты

Когда нота начинается, программа запоминает время и её длительность. Затем на каждом проходе сравнивает текущее время с началом.

```
прошло = сейчас - время_начала
если прошло >= длительность -> закончить ноту
```

Например, нота началась при значении счётчика 5000 мс и длится 1000 мс. На отметке 6000 мс её время закончится. Частота, например 440 Гц, влияет на высоту звука, а не на эти 1000 мс.

Одна нота и гамма

После одиночной ноты программа переходит к тишине. В гамме она выбирает следующую ноту. Здесь под гаммой понимаются **семь нот до, ре, ми, фа, соль, ля, си**, без заданных пауз между ними.

При времени 1 с на ноту вся последовательность занимает около 7 с при регулярном выполнении программы. Восьмое до не добавляется.

Почему музыка идёт без новых клавиш

Функция `piano_update` вызывается в каждом проходе основного цикла, даже если UART ничего не прислал. Поэтому нота сама закончится, а гамма сама продолжится.

Если основной цикл задержался

Программа вычисляет, какая нота должна играть по первоначальному расписанию. Если несколько нот уже истекли, она пропускает их, а не начинает последовательно доигрывать после задержки.

Например, гамма началась в 0 мс, каждая нота по 1 с. На 2500 мс должна быть ми, третья нота. Если обновление пришло только сейчас, программа выберет ми. До обновления аппаратный сигнал сохранял прежнюю частоту.

Это называют автоматом

Программа хранит состояние: тишина, одиночная нота или последовательность. На основе времени и клавиш она меняет это состояние. Такой способ организации называется конечным автоматом.

9. Что будет, если менять настройки во время звука

Настройки для будущего и настройки текущей игры

В программе есть два набора значений. **Selected** - что выбрано для следующего запуска. **Active** - с чем уже запущено текущее воспроизведение.

Когда начинается нота или гамма, выбранная октава и длительность копируются в active. Так вся начатая гамма сохраняет одни настройки.

Действие	Что произойдёт
Во время Ля 440 Гц нажать +	Текущая Ля сохранит частоту. Следующая будет выше на октаву.
Во время ноты нажать A	Текущая нота сохранит исходную длительность. Следующая будет длиннее на 0,1 с.
Во время гаммы изменить октаву	Вся текущая гамма доиграет в прежней октаве. Новая игра возьмёт новую.
Во время гаммы нажать 7	Гамма заменится отдельной нотой си с выбранными сейчас настройками.
Во время ноты нажать Enter	Начнётся новая гамма с до.

Повторное нажатие той же ноты

Оно заново отсчитывает длительность от нового события. Если частота осталась той же, драйвер звука не перезаписывает таймер без необходимости.

Неверная клавиша

Например, x даёт сообщение об ошибке с кодом 0x78. Она не заменяет текущую музыку: нота или гамма продолжает своё расписание.

Это выбранные в проекте правила для ситуаций во время воспроизведения. Они явно зафиксированы в README и проверены тестами.

10. Что осталось от UART из ЛР 2

Приём идёт через прерывания

Используется USART1 на PA9/PA10, AF7, скорость 57600 8N1. Готовые функции HAL принимают и отправляют байты с прерываниями. В ЛР 3 пользователь не переключает I/P: в приложении выбран режим с прерываниями.

Клавиши разбираются сразу

В ЛР 2 собиралась целая строка до Enter. Здесь каждый байт уже означает действие. Поэтому отдельная нота запускается сразу, а Enter имеет собственный смысл - запуск семи нот.

CR и LF - два варианта окончания строки. Если терминал отправляет пару CRLF, следующий за CR символ LF подавляется и гамма запускается один раз.

Очереди дают данным подождать

Буфер **RX на 128 байт** хранит принятые символы. **TX на 512 байт** хранит сообщения, которые ещё предстоит отправить. Кольцевые очереди сохраняют порядок и не требуют сдвигать весь массив.

Консоль проверяет, есть ли минимум 192 байта свободного TX для эхо и полного ответа. Если места мало, новые клавиши разбираются позже, а уже запущенная музыка продолжает отсчитываться.

Что бывает при потере данных

```
ERROR: input lost; repeat the last key
```

Ошибка приёма или переполнение помечают ввод как потерянный. После сообщения можно сразу повторить нужную клавишу. Дожидаться Enter для завершения испорченной строки здесь не нужно: длинных команд нет.

Пределы очередей

Flow control не используется. Бесконечный быстрый поток нельзя сохранить в конечной памяти. Для демонстрации ориентируйся на ответы терминала. При удержании клавиши терминал может посылать автоповторы, каждый будет отдельным действием.

11. Как читать основные файлы

Файл	За что отвечает
main.c	Соединяет все части в основном цикле
piano.c	Выбирает ноту, октаву, длительность и ход гаммы
sound.c	Переводит частоту в настройки TIM2 и включает звук
clock.c	Настраивает 120 МГц и частоты шин
console.c	Обрабатывает клавиши и формирует текстовые ответы
serial.c и uart_hal.c	Принимают и отправляют байты через UART
board.h	Содержит вывод PA5, AF1 и основные частоты

Четыре шага основного цикла

```
serial_service();
piano_update(&piano, now);
console_update(&console, &piano, now);
sound_set_frequency(piano_frequency_millihz(&piano));
```

Сначала обслуживается передача UART. Затем обновляется музыка по времени. Потом обрабатываются клавиши. В конце текущая частота передаётся звуковому драйверу. Значение now получается из HAL_GetTick.

Почему piano не настраивает TIM2 напрямую

Piano знает музыкальные правила, но не устройство платы. Его результат - частота в миллигерцах или ноль для тишины. Sound знает регистры таймера, но не названия нот и клавиши.

Это позволяет проверять музыкальную логику отдельно и менять звуковую аппаратуру, сохраняя правила клавиатуры.

Почему нет HAL_Delay

Главный цикл должен продолжать принимать клавиши и следить за временем. Длинная задержка до конца ноты мешала бы этому. Поэтому ожидание хранится как время начала и нужная длительность.

Вопросы 1-10. Что сделано и как управлять

Ответы короткие: их удобно объяснять своими словами.

1. Что ты сделал в ЛР 3?

Музыкальную клавиатуру на стенде. Компьютер отправляет клавиши по UART, а микроконтроллер выбирает ноту и формирует сигнал для звукоизлучателя.

2. Чем работа отличается от ЛР 2?

Вместо управления светофором здесь управление музыкой. От ЛР 2 сохранён UART с очередями, а добавлены таймер звука и музыкальная логика.

3. Почему при номере 6 реализован вариант 1?

В README проекта принят циклический выбор из пяти вариантов: (6 - 1) % 5 + 1 = 1. Здесь этот вариант - музыкальная клавиатура.

4. Что выбрано после сброса?

Звука нет, выбрана первая октава 5/9 и длительность одной ноты 1,0 с.

5. Что делают клавиши 1-7?

Запускают до, ре, ми, фа, соль, ля, си выбранной октавы. Каждое нажатие выбирает одну ноту.

6. Что делают плюс и минус?

Выбирают октаву выше или ниже. Допустимы номера от 1 до 9.

7. Что делают A и a?

Большая латинская A увеличивает длительность на 0,1 с, маленькая a уменьшает. Это две разные команды.

8. Какие пределы длительности?

От 0,1 до 5,0 с. Попытка выйти за границу даёт LIMIT и сохраняет прежнее значение.

9. Что делает Enter?

Запускает семь нот до, ре, ми, фа, соль, ля, си подряд, с одной выбранной длительностью для каждой.

10. Что будет от неподдерживаемой клавиши?

Появится ERROR с кодом символа. Текущая музыка не заменяется ошибочным вводом и продолжает идти по времени.

Вопросы 11-20. Клавиатура и текущая музыка

11. Нужно ли после цифры нажимать Enter?

Нет. Цифра должна отправляться и обрабатываться сразу. Enter запускает гамму.

12. Зачем выключать Local line editing?

Иначе терминал может хранить нажатия до Enter. Тогда программа не получит клавишу в нужный момент.

13. Зачем UTF-8 и английская раскладка?

UTF-8 нужен для русских сообщений об ошибках. Управляющие символы A/a должны быть латинскими, поэтому для них нужна английская раскладка.

14. Какие настройки UART используются?

USART1, PA9/PA10, AF7, 57600 8N1, без flow control. В приложении UART работает с прерываниями через HAL.

15. Почему нужен SA2.7?

Он соединяет сигнал таймера с усилителем звукоизлучателя. При разомкнутой цепи на PA5 сигнал может быть, но до BA1 он не дойдёт.

16. Что будет, если во время ноты сменить октаву?

Текущая нота сохранит прежнюю октаву. Новая настройка используется при следующем запуске.

17. А если изменить длительность во время гаммы?

Вся текущая гамма сохранит прежнюю длительность каждой ноты. Следующая нота или новая гамма возьмёт новое значение.

18. Что будет от новой цифры во время гаммы?

Она заменит гамму отдельной нотой. Новый Enter тоже заменяет текущую игру: запускает гамму с начала.

19. Почему CRLF не запускает гамму дважды?

После CR программа игнорирует сразу следующий LF. Одна пара CRLF считается одним Enter.

20. Если держать клавишу, нота всегда будет только одна?

Не обязательно. Терминал может отправлять автоповторы. Каждый полученный символ заново запускает действие, в том числе отсчёт длительности той же ноты.

Вопросы 21-30. Ноты и частоты

21. Что такое частота звука в этой работе?

Количество повторений электрического сигнала за секунду. Она задаёт основную высоту ноты и измеряется в герцах.

22. Что будет, если увеличить частоту в два раза?

Та же нота прозвучит на октаву выше. Например, Ля 440 Гц станет Ля 880 Гц.

23. Как перейти на октаву ниже?

Уменьшить частоту вдвое. Например, из 440 Гц получится 220 Гц.

24. Почему первая октава - номер 5?

Список начинается с более низких октав: субконтроктавы, контроктавы, большой и малой. После них идёт первая, пятая по счёту в списке.

25. Есть ли здесь все двенадцать звуков октавы?

Нет. Клавиши задают семь натуральных нот до-си. Для расчёта частот используется музыкальная шкала, в которой октава делится на 12 полутонов.

26. Сколько частот записано в таблице?

63: семь нот в каждой из девяти октав.

27. Что означает 440000 в таблице?

440000 миллигерц, то есть 440 Гц. В одном герце тысяча миллигерц.

28. Откуда взяты частоты и зачем хранить их в таблице?

Частоты заранее рассчитаны и округлены до одного миллигерца. Формула: f = 440 × 2^(k/12), где k - число полутонов от Ля 440 Гц. При k=12 получаем 880 Гц. Таблица избавляет от этих вычислений во время игры.

29. Какой клавишей проверить Ля 440 Гц?

После сброса нажать 6: выбрана первая октава, и шестая нота в ряду - Ля.

30. Клавиша A увеличивает громкость?

Нет, она увеличивает время звучания на 0,1 с. Отдельного управления громкостью в этой реализации нет.

Вопросы 31-40. Таймер и ШИМ

31. Что такое аппаратный таймер?

Устройство внутри микроконтроллера, которое считает такты и может само формировать сигнал. Процессору не нужно вручную делать каждый переход между 0 и 1.

32. Какой таймер и вывод используются?

TIM2, канал 1. Он выведен на PA5 через AF1. Дальше сигнал проходит через SA2.7 и усилитель к BA1.

33. Что такое ШИМ, или PWM?

Повторяющийся сигнал с высоким и низким участками. Таймер позволяет задать период и длительность высокого участка.

34. Как работает PWM mode 1 в этом коде?

Пока CNT меньше CCR1, выход высокий. В оставшейся части периода низкий. Счёт направлен вверх.

35. Для чего PSC?

Это предделитель тактов таймера. Частота счёта делится на PSC+1; здесь PSC=0, поэтому деления скорости нет.

36. Для чего ARR?

Это последнее число счётчика в периоде. В периоде ARR+1 тактов, потому что отсчёт начинается с нуля.

37. Для чего CCR1?

Он задаёт границу высокого участка первого канала. Здесь примерно половина периода проходит в высоком уровне.

38. Почему при ARR=9 получается 10 тактов?

Счётчик проходит десять значений: от 0 до 9 включительно. Поэтому в формуле всегда используется ARR+1.

39. Нужно ли прерывание на каждый импульс звука?

Нет. Импульсы создаёт аппаратура TIM2. В коде DIER=0, прерывания этого таймера не включены.

40. Зачем 32-разрядный TIM2?

Он позволяет вместить длинные периоды низких нот без изменения PSC. На 16-битном таймере можно было бы подобрать другой делитель, но здесь это не требуется.

Вопросы 41-50. Расчёты и остановка

41. По какой формуле определяется частота PWM?

Частота таймера делится на (PSC+1) и на (ARR+1). Здесь это 60000000 / (ARR+1), потому что PSC=0.

42. Какую частоту подставлять для TIM2?

60 МГц. Нельзя подставлять частоту ядра 120 МГц или частоту APB1 30 МГц без учёта настройки таймера.

43. Сколько тактов нужно на период 440 Гц?

Примерно 60000000 / 440. После округления получается 136364 такта.

44. Чему равен ARR для Ля 440 Гц?

136363: на единицу меньше числа тактов в периоде.

45. Чему равен CCR1 и что значит заполнение?

CCR1=68182, половина периода. Заполнение - доля высокого участка, около 50%. Скважность - обратное отношение, период к высокому участку: около 2.

46. Почему расчётная частота чуть отличается от 440 Гц?

Период можно задать только целым числом тактов. После округления выходит около 439,9988 Гц. Погрешность HSI учитывается отдельно.

47. PSC=0 означает остановку?

Нет, это деление на PSC+1, то есть на единицу. Остановка счёта управляется битом CEN.

48. Зачем uint64_t в формуле периода?

Промежуточное число 60000000 × 1000 равно 60 миллиардам. Оно не помещается в 32-битный uint32_t, но помещается в 64-битный тип.

49. Что делают предзагрузка и UG?

Новые ARR и CCR1 сначала сохраняются в подготовительных регистрах. Событие UG переносит их в рабочие значения до старта счётчика. Это обеспечивает запуск с нужным периодом.

50. Как выключается звук?

CCR1 устанавливается в ноль, настройки загружаются через UG, счёт выключается через CEN, CNT обнуляется. Канал остаётся включённым в PWM1, чтобы PA5 удерживался низким.

Вопросы 51-60. Частоты микроконтроллера

51. С какой частотой работает ядро?

120 МГц. Это частота процессора, а не частота звука и не частота всех периферийных устройств.

52. Откуда берётся исходная частота?

Из внутреннего генератора HSI 16 МГц. Программа не требует внешнего кварца для своего источника тактирования.

53. Что делает PLL?

Получает из исходной частоты другие частоты с помощью делителей и умножителя. Здесь 16 / 8 × 120 / 2 = 120 МГц.

54. Какие частоты у APB1 и APB2?

APB1 - 30 МГц, деление ядра на 4. APB2 - 60 МГц, деление на 2. Это шины, к которым подключена периферия.

55. Почему TIM2 получает 60 МГц от APB1 на 30 МГц?

TIMPRE=0 и делитель APB1 больше единицы. При этих условиях таймер получает удвоенную частоту своей шины.

56. USART1 тоже получает удвоенную частоту APB2?

Нет. USART1 использует 60 МГц своей шины. Правило удвоения здесь относится к таймерам.

57. Почему BRR UART уже не 278, как в ЛР 2?

Частота APB2 выросла с 16 до 60 МГц. Для 57600 бод HAL рассчитывает новый делитель BRR=1042, то есть 0x412.

58. Почему SysTick остался миллисекундным?

HAL перенастраивает его после изменения HCLK. При 120 МГц LOAD=119999 даёт 120000 тактов на период 1 мс.

59. Зачем Flash latency 3?

При высокой частоте чтению Flash нужны такты ожидания. В проекте для 120 МГц и питания стенда выбрано три. Это настройка памяти, а не длительность ноты.

60. Зачем сначала переключаться на HSI при настройке PLL?

Отладчик мог оставить другую работающую PLL. Код сначала переходит на HSI, затем спокойно задаёт нужную PLL и возвращает системную частоту на неё.

Вопросы 61-70. Время и музыкальная логика

61. Зачем одновременно TIM2 и SysTick?

TIM2 задаёт быстрые импульсы нужной высоты. SysTick даёт миллисекунды для длительности нот и переходов гаммы.

62. Как программа понимает, что нота закончилась?

Вычитает из текущего времени время начала и сравнивает с сохранённой длительностью. Если время вышло, завершает ноту или переключает гамму.

63. Почему не поставить задержку на одну секунду?

Длинная пауза в основном цикле мешала бы обработке клавиш. Вместо ожидания программа хранит время начала и регулярно проверяет, сколько прошло.

64. Что будет при переполнении счётчика миллисекунд?

Он начнёт с нуля примерно через 49,7 суток. Беззнаковая разность now - started корректна для наших интервалов при регулярных обновлениях, даже через этот переход.

65. Гамма по 1 с началась в 0. Какая нота на 2500 мс?

Ми, третья нота: до идёт 0-1 с, ре 1-2 с, ми 2-3 с. При позднем обновлении программа сразу выбирает соответствующую ноту.

66. Что будет на отметке 7 секунд такой гаммы?

Время семи нот закончится, программа перейдёт к тишине. Реальная смена выхода происходит при обслуживании основного цикла.

67. Добавляется ли после си ещё одно до или пауза?

Нет. В этой реализации играют ровно семь нот выбранной октавы. Отдельные паузы между ними не заданы.

68. Может ли неверная клавиша продлить текущую ноту?

Нет. Ошибка ввода сама не перезапускает ноту. При обработке сначала обновляется время, затем выдаётся результат ошибки.

69. Почему одинаковую частоту не записывают в TIM2 каждый проход?

Драйвер хранит current_frequency. Если частота уже выставлена, он возвращается без перенастройки. При новом нажатии той же ноты её время всё равно начинается заново.

70. Что будет при недопустимой частоте в sound_set_frequency?

Функция вернёт false и оставит предыдущий звук без изменения. Ноль обрабатывается отдельно: он означает тишину.

Вопросы 71-80. Код, UART и проверка

71. Почему piano.c отделён от sound.c?

Piano решает, какую ноту играть и когда остановиться. Sound принимает частоту и настраивает аппаратуру. Музыкальные правила можно проверять отдельно от платы.

72. Что делает главный цикл?

Обслуживает UART, обновляет музыку по времени, разбирает клавиши и передаёт текущую частоту звуковому драйверу. Длинных ожиданий в нём нет.

73. Зачем очереди RX и TX?

RX на 128 байт хранит принятые клавиши. TX на 512 байт - ответы для отправки. Это помогает обмену и музыкальной логике работать без ожидания друг друга.

74. Что происходит после потери ввода?

Появляется просьба повторить последнюю клавишу. Повтор можно отправить сразу: ждать Enter для восстановления не нужно, потому что действия однобайтовые.

75. Если очередь TX занята, звук остановится?

Нет. Консоль откладывает новые клавиши при нехватке места, а piano_update продолжает обслуживать текущую музыку. При неограниченном потоке без flow control возможны потери RX.

76. Как избегают конфликтов main и UART-прерывания?

Для общих данных очереди используются короткие защищённые участки и восстановление прежнего запрета прерываний. API рассчитан на один main и один обработчик USART1.

77. Что проверяют программные тесты?

Все 63 частоты, клавиши, пределы, длительность, гамму, регистры TIM2, часы, UART и запуск готовой прошивки. Они исполняют скомпилированный ARM-код.

78. Что тесты не могут подтвердить без стенда?

Реальное звучание, прохождение сигнала через усилитель, USB/COM и фактическую точность HSI. В модели проверяются код и обращения к регистрам.

79. Что проверить, если ответы есть, а звука нет?

Замкнут ли SA2.7, включён ли путь PA5/AF1 и TIM2_CH1, есть ли питание. Затем - CEN, ARR, CCR1 и lab3_fault. Если есть осциллограф, посмотреть сигнал на PA5.

80. Что показать для Ля 440 Гц?

После сброса нажать 6. В терминале должны быть 440.000 Hz и 1.0 s; у TIM2 PSC=0, ARR=136363, CCR1=68182. После ноты CCR1=0 и CEN=0.

Практика 1. Что произойдёт от клавиш

Время в примерах - время, когда программа обработала полученную клавишу.

Ситуация	Правильный ответ
После сброса нажать 6	Ля первой октавы, 440 Гц, на 1 с
Нажать +, затем 6	Ля второй октавы, 880 Гц, на 1 с
Нажать A три раза после сброса	Для следующего запуска выбрана длительность 1,3 с
Снизить октаву до 1 и ещё нажать -	Остаётся 1, в ответе LIMIT
Довести длительность до 5 с и нажать A	Остаётся 5 с, в ответе LIMIT
Во время ноты нажать x	Сообщение об ошибке с 0x78; нота сохраняет своё расписание
Во время гаммы нажать 3	Вместо гаммы начинается отдельная ми
Получить CR и сразу LF	Одна гамма, а не две

Задача с изменением настроек

В 0 мс нажали 6 при 440 Гц и 1 с. В 200 мс нажали +, в 300 мс - A. Что слышно и что будет дальше?

Ответ: исходная Ля сохраняет 440 Гц и заканчивается примерно на отметке 1000 мс при обычном выполнении. Для нового запуска выбраны вторая октава и 1,1 с. Следующая клавиша 6 даст 880 Гц на 1,1 с.

Задача с запоздавшим обновлением

Гамма началась в 0 мс, по 1 с на ноту. После долгой задержки основной цикл вернулся в 4500 мс. Выбирается соль, пятая нота. Пропущенные ре, ми и фа не добавляются потом в конец последовательности.

Практика 2. Числа, которые стоит понимать

Вопрос	Ответ
Что означает 440000 mHz?	440 Гц, то есть Ля первой октавы
Что означает 1000 ms?	Одна секунда длительности
Что означает 60 MHz?	60 миллионов тактов таймера в секунду
Что значит первая октава 5/9?	Пятая позиция в списке из девяти октав
Почему ARR на 1 меньше периода в тактах?	Потому что счётчик начинает с нуля
Почему CCR1 примерно половина ARR+1?	Высокий уровень должен занимать около половины периода

Расчёт с маленькими числами

Пусть на таймер условно приходит 1000 тактов в секунду, PSC=0 и ARR=9. Частота будет 1000 / 10 = 100 Гц. Если CCR1=5, заполнение равно 5/10 = 50%.

Если оставить частоту счёта 1000, но задать ARR=19, получится 1000/20 = 50 Гц. Более длинный период даёт меньшую частоту. Это учебные числа для понимания, а не настройки прошивки.

Расчёт для реальной платы

```
TIM2CLK = 60 000 000 Hz
PSC = 0
K = round(60 000 000 / 440) = 136364
ARR = 136363
CCR1 = 68182
```

Проверка: 60000000 / 136364 ≈ 439,9988 Гц. При переходе к 880 Гц период должен стать примерно вдвое короче.

Вопрос с подвохом

Ядро работает на 120 МГц. Нужно ли подставлять 120 миллионов в формулу TIM2? Нет: важна собственная частота таймера. Здесь это 60 МГц после делителей шин и удвоения для TIM2.

Что показать и повторить перед защитой

Короткий маршрут демонстрации

1. Подключение. Питание, DBG USB, SA2.7 замкнут. Терминал 57600 8N1, UTF-8, эхо и редактирование строки в терминале выключены.

2. Одна нота. После сброса нажать 6 без Enter. Показать ответ с 440 Гц, звук и завершение через 1 с.

3. Настройки. Показать +/-, A/a и LIMIT на границах. Объяснить разницу высоты и длительности.

4. Гамма. Нажать Enter, изменить настройки во время игры, показать сохранение настроек текущей гаммы. Затем заменить её отдельной нотой.

5. Ошибка. Нажать x, увидеть код 0x78, затем сразу правильную клавишу. Не нужно восстанавливать ввод через Enter.

6. Регистры. Для Ля первой октавы показать PSC=0, ARR=136363, CCR1=68182. После окончания - CCR1=0, CEN=0. При наличии осциллографа измерить PA5.

Пять фраз, которые нужно объяснить уверенно

TIM2 создаёт частоту. SysTick помогает отсчитать длительность. Октава выше удваивает частоту. ARR задаёт период, CCR1 - высокий участок. Новые настройки действуют со следующего запуска.

Как собрать и прошить

```
python3 lab3/tools/build.py
python3 lab3/tools/board.py flash
```

Из корня проекта. Результат сборки - `lab3/build/lab3.elf` и `.bin`. Используется FT2232/JTAG с SDK11M.cfg. В CubeIDE это самостоятельный проект lab3.

Длительности проверяют при свободном выполнении, без остановок отладчика. Реальный источник HSI имеет погрешность, а крайние ноты могут быть хуже слышны из-за свойств звукоизлучателя.

Карта проекта, источники и проверка

Что искать	Файл внутри lab3
Клавиши, ноты и настройки	Core/Inc/piano.h, Core/Src/piano.c
Настройка и остановка TIM2	Core/Src/sound.c
PLL и частоты шин	Core/Src/clock.c
Основной цикл	Core/Src/main.c
Ответы и сообщения гаммы	Core/Src/console.c
Очереди и аппаратный UART	Core/Src/serial.c, uart_hal.c, uart_pins.c
Схема звукового тракта	docs/SDK1.1M.pdf, страницы PDF 2-4 и 7
Тесты	tests/test_lab3.c, tests/run_tests.py

Что проверено при подготовке

03.11.2026 прошивка и тестовая сборка собраны в отдельной временной копии проекта. Прошли **370 проверок С-кода и 313 проверок таймера, UART и запуска**. Включён запуск готовой прошивки от Reset_Handler.

Модель проверяет работу кода и регистры, но не воспроизводит физический звук, усилитель и USB. Реальная работа стенда в этой подготовке не подтверждалась.

Основа памятки

Исходники lab3, README.md, docs/DEFENSE.md, docs/VALIDATION.md и электрическая схема. Памятка следует текущему коду и правилам, записанным в проекте; это не официальный список вопросов к защите.

README ссылается на пособие SDK-1.1M редакции 23.09.2026, где раздел задания отмечен как уточняемый. Поведение новых клавиш, пределы и политика смены воспроизведения здесь описаны по реализации проекта.

RM0090: таймеры, PWM и тактирование STM32F4

DS9405: STM32F427, выводы и допустимые частоты